

relative x-axis. The `turtle.left(90)` call points the turtle along the turtle's relative y-axis, and `turtle.forward(size * yCh)` moves the turtle to the next shape's starting position. However, the `turtle.left(90)` call changed the turtle's heading, so `turtle.right(90)` is called to reset it back to its original direction.

[Figure 13-10](#) shows how these four lines of code move the turtle to the right along its relative x-axis and up along its relative y-axis and leave it in the correct heading, no matter what its initial heading was.

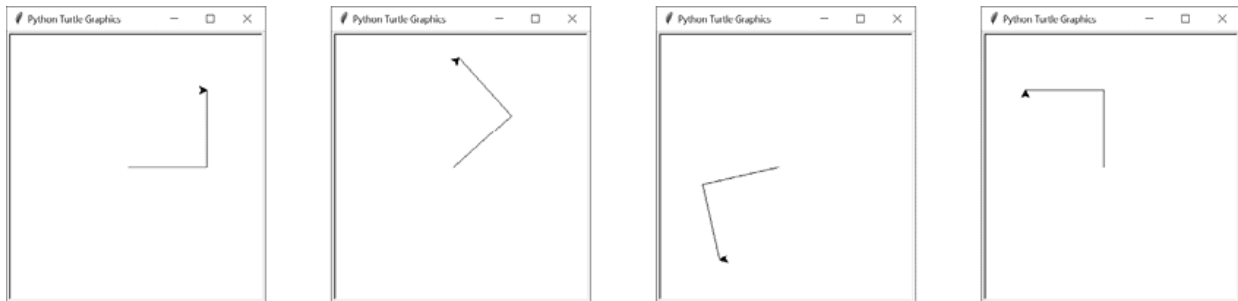


Figure 13-10: In each of these four images, the turtle always moves 100 units “right” and “up” along the relative x-axis and y-axis of its initial heading.

Finally, with the turtle in the correct position and heading for the next shape, we make the recursive call to `drawFractal()`:

Python

```
# Make the recursive call:
drawFractal(shapeDrawFunction, size * sizeCh, specs, maxDepth,
depth + 1)
```

The `shapeDrawFunction`, `specs`, and `maxDepth` arguments are passed to the recursive `drawFractal()` call unchanged. However, `size * sizeCh` is passed for the next `size` parameter to reflect the change in the `size` of the recursive shape, and `depth + 1` is passed for the `depth` parameter to increment it for the next shape-drawing function call.